

EE1021 – Lecture 9 Home Worksheet

Topics: object-oriented programming (classes, instances, attributes, methods); special methods (`__init__`, `__str__`, comparisons, arithmetic); properties; class vs instance variables; composition.

Tip: Unless a task explicitly says “print”, prefer returning values from methods. You can print returned values when testing.

Part A – Class Design & Methods

1. Battery – State and Behavior

Create a class `Battery` with the following behavior:

- Attributes:
 - `capacity_mAh`: total capacity in mAh.
 - `charge_mAh`: current charge in mAh (default 0).
- Methods:
 - `charge(self, mAh)`:
 - * Increases `charge_mAh` by `mAh`.
 - * Never allow `charge_mAh` to exceed `capacity_mAh` (clamp to capacity).
 - * Negative or zero values for `mAh` should raise `ValueError`.
 - `consume(self, current_mA, hours)`:
 - * Reduces `charge_mAh` by `current_mA * hours`.
 - * Never allow `charge_mAh` to go below 0 (clamp to 0).
 - * Negative or zero values for `current_mA` or `hours` should raise `ValueError`.
 - `level_pct(self)`:
 - * Returns the current percentage as a float in `[0.0, 100.0]`.
 - `__str__(self)`:
 - * Returns a string like "Battery 76% (2280/3000 mAh)", where the percentage is rounded to the nearest integer.

2. Thermostat – Validation and Simple Logic

Create a class `Thermostat` with:

- Attributes:

- `setpoint`: a float (target temperature in °C).
- `mode`: a string in {"heat", "cool", "off"}.
- `tolerance`: a float (default 0.5, in °C).
- Methods:
 - `set_mode(self, mode)`:
 - * Updates mode.
 - * If mode is not one of "heat", "cool", or "off", raise `ValueError`.
 - `needs_action(self, current_temp)`:
 - * Returns:
 - "heat_on" if mode == "heat" and current_temp < setpoint - tolerance.
 - "cool_on" if mode == "cool" and current_temp > setpoint + tolerance.
 - "idle" otherwise.
 - `__str__(self)`:
 - * Returns a string like `Thermostat(mode='heat', setpoint=22.0, tol=0.5)`.

Part B – Special Methods & Value Types

3. ColorRGB – String, Addition, Clamp

Create a class `ColorRGB(r, g, b)` representing an RGB color:

- Initialization:
 - Each channel (r, g, b) should be stored as an integer in [0, 255].
 - If the provided value is outside [0, 255], clamp it to that range (do **not** raise an error).
- Methods:
 - `to_hex(self)`:
 - * Returns a string like "#1a2b3c":
 - Lowercase hex.
 - Two hex digits per channel.
 - `__str__(self)`:
 - * Returns the same value as `to_hex()`.
 - `__add__(self, other)`:
 - * Returns a new `ColorRGB` where each channel is the average of `self` and `other`:
 - `r = round((self.r + other.r) / 2)` (similarly for g, b).
 - `lighter(self, percent)`:
 - * Returns a new `ColorRGB` that is lightened by `percent`.
 - * For example, `percent = 20` means “move 20% of the way from the current value toward 255” for each channel.
 - * Clamp each result to 255.
 - `__eq__(self, other)`:
 - * Two colors are equal if all three channels are equal.

4. TimeSpan – Normalization and Operators

Create a class `TimeSpan(hours=0, minutes=0, seconds=0)` representing a non-negative duration:

- Internals:
 - Store the duration in a normalized, non-negative form (e.g., only total seconds, or normalized `h:m:s`).
 - For example, `TimeSpan(minutes=90)` should be equivalent to `TimeSpan(hours=1, minutes=30)`.
- Methods:
 - `total_seconds(self)`:
 - * Returns the total duration as an `int` (number of seconds).
 - `__add__(self, other)`:
 - * Returns a new `TimeSpan` equal to the sum of the two durations.
 - `__sub__(self, other)`:
 - * Returns a new `TimeSpan` equal to the difference.
 - * If the result would be negative, clamp to zero (i.e., return a zero `TimeSpan`).
 - Comparisons:
 - * Implement `__eq__(self, other)` and `__lt__(self, other)` based on `total_seconds()`.
 - `__str__(self)`:
 - * Returns a string in the format `"HH:MM:SS"` with zero-padded fields.
- Class method:
 - `@classmethod from_seconds(cls, n)`:
 - * Builds and returns a `TimeSpan` from `n` seconds (non-negative integer).

Part C – Containers & Composition (No Inheritance)

5. SlidingWindow – Running Statistics

Create a class `SlidingWindow(size)` that keeps the most recent values:

- Initialization:
 - `size` is the maximum number of values to remember.
 - If `size <= 0`, raise `ValueError`.
- Behavior:
 - Internally store up to `size` numeric values in order (oldest first).
- Methods:
 - `add(self, x)`:
 - * Appends a new value `x`.
 - * If the window would exceed `size`, drop the oldest value.
 - `mean(self)`:
 - * Returns the average of the current values.
 - * If the window is empty, raise `ValueError`.

- `min(self)` and `max(self)`:
 - * Return the minimum / maximum of the stored values.
 - * If the window is empty, raise `ValueError`.
- `__len__(self)`:
 - * Returns the number of stored values.
- `__str__(self)`:
 - * Returns a compact summary like `"SlidingWindow(n=3, values=[1.0, 2.0, 3.0])"`.

6. Library – Tracking Loans

You will use composition (no inheritance) to model a simple library.

- Class `Member`:
 - `Member(member_id, name)`:
 - * Stores:
 - `member_id` (an integer).
 - `name` (a string).
 - A collection of borrowed book titles (strings).
 - Methods:
 - * `borrow(self, title)`:
 - Adds `title` to the member's borrowed collection.
 - Ignore duplicates (a title should appear at most once per member).
 - * `return_book(self, title)`:
 - Removes `title` if it is currently borrowed.
 - If it is not present, do nothing.
 - * `borrowed_count(self)`:
 - Returns the number of borrowed books.
- Class `Library`:
 - `Library()`:
 - * Tracks members (you may store them in a list or dictionary).
 - * Maintains an internal counter used to assign each new member a unique, incremental ID.
 - * Store this counter as an **instance attribute**, not a class attribute.
 - Methods:
 - * `enroll(self, name)`:
 - Creates a new `Member` with the next available ID.
 - Stores the member inside the `Library`.
 - Returns the new `Member` instance.
 - * `find_by_id(self, member_id)`:
 - Returns the `Member` with the given `member_id`, or `None` if not found.
 - * `total_borrowed(self)`:
 - Returns the total number of borrowed books across all mem-

bers.

Deliverables

- Submit a single `.py` file containing:
 - All the classes defined above.
 - Minimal test code (a few simple examples) that demonstrates each class works.
- You may also use the provided template: `Lecture9/EE1021_Lecture9_HomeWorksheet_Template.py`.
- Ensure that invalid inputs raise `ValueError` where specified, and that your code runs without errors.